

Performance Analysis of OpenMP Parallelization for Numerical Integration

Dheeraj Sai Vulchi

I. INTRODUCTION

High performance computing systems rely on parallel programming to improve execution time. In this assignment, OpenMP was used to parallelize a numerical integration program that approximates the value of π . The goal of this study was to evaluate the impact of OpenMP runtime parameters such as thread placement, number of threads, and scheduling policies on application performance on the Lonestar6 cluster.

II. SYSTEM ARCHITECTURE

Experiments were performed on a Lonestar6 compute node with the following characteristics:

- Processor: AMD EPYC 7763
- Number of sockets: 2
- Cores per socket: 64
- Total cores: 128
- NUMA nodes: 2
- L1 cache: 32 KB
- L2 cache: 512 KB
- L3 cache: 32 MB
- Total memory: approximately 251 GB

This architecture provides a large number of cores for evaluating OpenMP scalability.

III. PARALLEL IMPLEMENTATION

The sequential loop computing the numerical integration was parallelized using OpenMP.

The following constructs were used:

- **parallel for**: distributes loop iterations across threads
- **reduction(+:sum)**: combines thread results safely
- **schedule(runtime)**: allows scheduling policies to be controlled using environment variables

Variable classification:

- **Private**: loop index i , temporary variable x
- **Shared**: step size and number of iterations
- **Reduction**: sum

IV. THREAD PLACEMENT RESULTS

The application was executed using different OpenMP thread placement and binding policies.

Observations:

- The best performance occurred when using `OMP_PLACES=cores` and `OMP_PROC_BIND=close`.
- Keeping threads close to cores improves cache locality.
- Socket-based placement slightly increases memory latency.

OMP_PLACES	OMP_PROC_BIND	Time (s)
cores	close	0.267533
cores	spread	0.269385
sockets	close	0.293718
sockets	spread	0.290274

TABLE I
THREAD PLACEMENT PERFORMANCE USING 128 THREADS

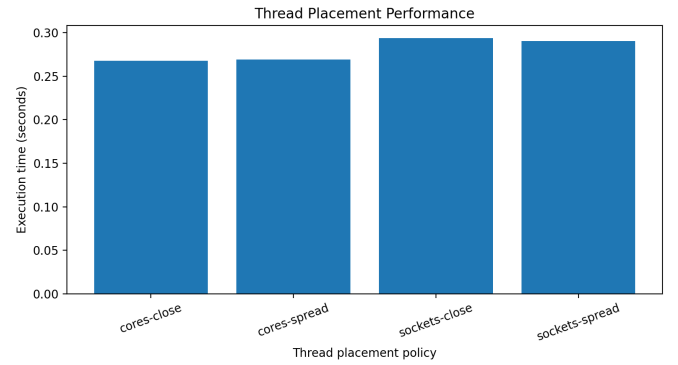


Fig. 1. Execution time for different OpenMP thread placement policies

V. THREAD SCALING

The number of threads was varied from 1 to 128 using the best placement policy.

Threads	Time (s)
1	0.013932
2	0.007007
4	0.003571
8	0.002076
16	0.001312
32	0.001466
64	0.001773
128	0.005394

TABLE II
EXECUTION TIME FOR DIFFERENT THREAD COUNTS

Observations:

- Performance improves significantly from 1 to 16 threads.
- The best execution time occurs at 16 threads.
- After 32 threads, overhead and memory contention reduce scalability.
- Using 128 threads increases overhead and results in slower performance.

VI. SCHEDULING POLICIES

Static and dynamic scheduling policies were compared using chunk sizes of 10, 100, and 1000.

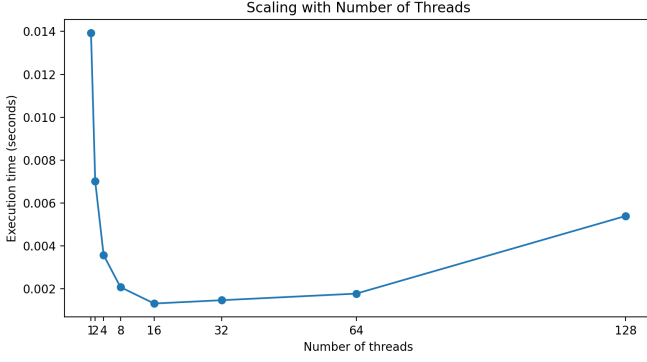


Fig. 2. Performance scaling as the number of threads increases

Schedule	Chunk Size	Time (s)
static	10	0.007970
static	100	0.002701
static	1000	0.009894
dynamic	10	0.033190
dynamic	100	0.005938
dynamic	1000	0.003140

TABLE III
SCHEDULING POLICY COMPARISON

Observations:

- Static scheduling performs best because the workload is evenly distributed.
- Dynamic scheduling introduces additional runtime overhead.
- The best configuration observed was static scheduling with chunk size 100.

VII. CONCLUSION

The OpenMP implementation significantly improved the performance of the numerical integration program. Thread placement close to cores provided slightly better performance due to improved cache locality. Performance scaled well up to 16 threads, after which overhead reduced the benefits of additional threads. Among the scheduling policies, static scheduling with moderate chunk sizes produced the best performance.

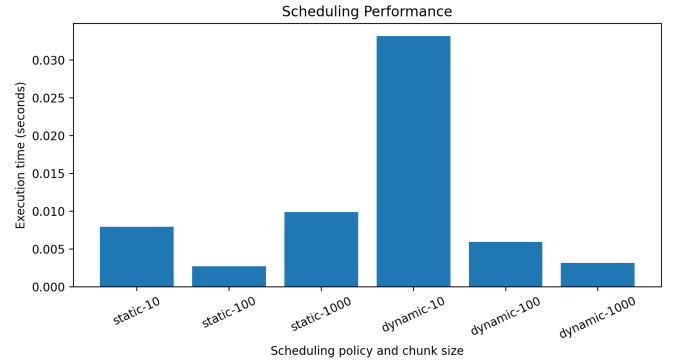


Fig. 3. Execution time for static and dynamic scheduling